

EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from

© Pearson Education

Hengzhao Yang

Zhifeng Zhu

Yajun Ha

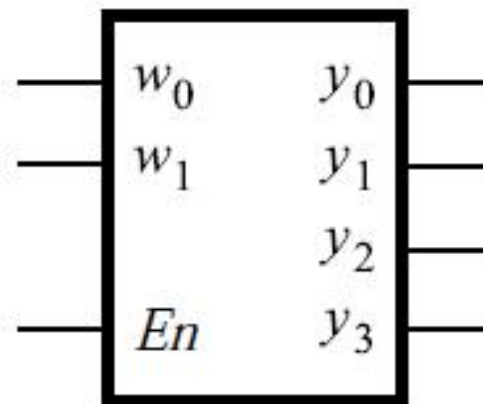


上海科技大学
ShanghaiTech University

Review

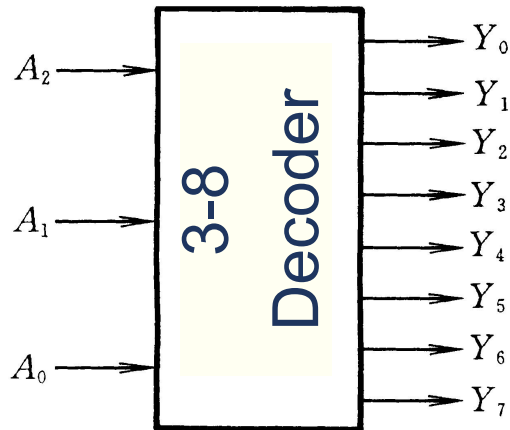
- Detects the presence of a specified combination of bits (code) on its inputs and indicates the presence of that code by a specified output level.

E_n	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0



2-4 Decoder with EN input

3-8 Binary Decoder



$$Y_0 = A_2' A_1' A_0' = m_0$$

$$Y_1 = A_2' A_1' A_0 = m_1$$

$$Y_2 = A_2' A_1 A_0' = m_2$$

...

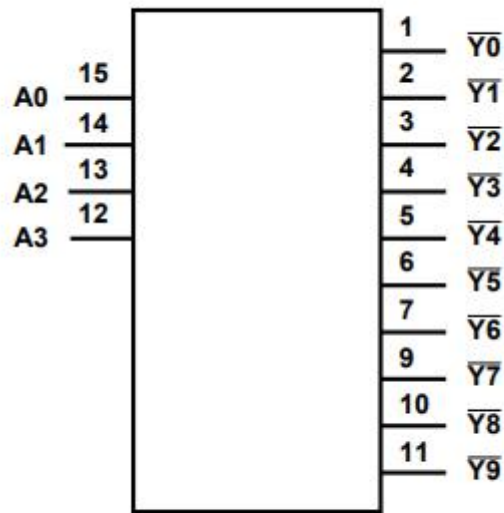
$$Y_7 = A_2 A_1 A_0 = m_7$$

输 入			输 出							
A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

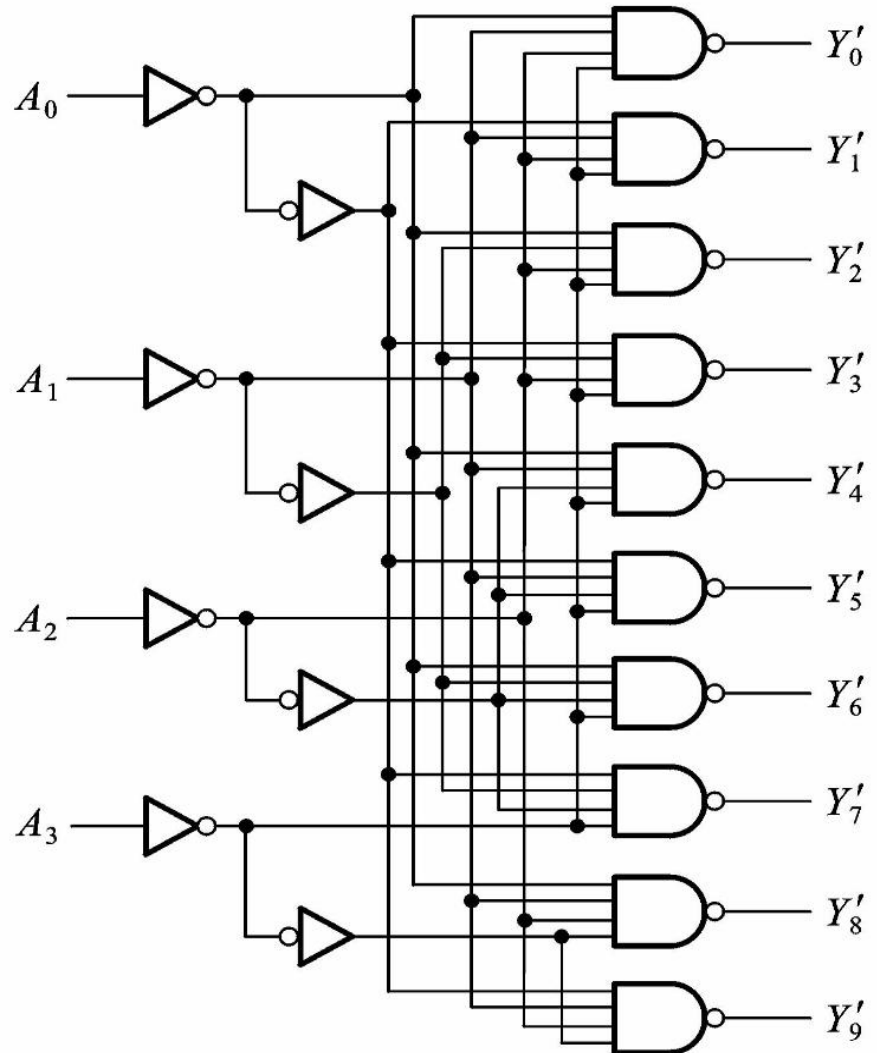


4-10 Binary Decoder (74HC42)

$$Y'_i = m'_i \quad (i = 0 \sim 9)$$

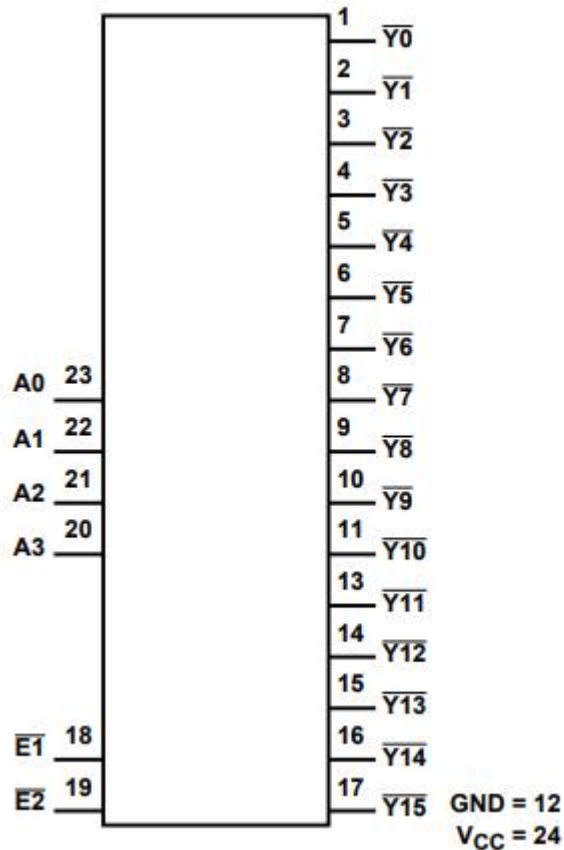


<https://www.ti.com/product/CD74HC42>



4-line to 16 Line Decoder

Output: active LOW



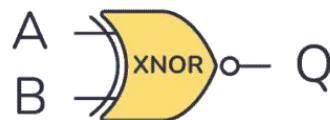
INPUTS						OUTPUTS															
E1	E2	A3	A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7	Y8	Y9	Y10	Y11	Y12	Y13	Y14	Y15
L	L	L	L	L	L	L	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H
L	L	L	L	L	H	H	L	H	H	H	H	H	H	H	H	H	H	H	H	H	H
L	L	L	L	H	L	H	H	L	H	H	H	H	H	H	H	H	H	H	H	H	H
L	L	L	L	H	H	H	H	H	L	H	H	H	H	H	H	H	H	H	H	H	H
L	L	L	H	L	L	H	H	H	H	L	H	H	H	H	H	H	H	H	H	H	H
L	L	L	H	L	H	H	H	H	H	H	L	H	H	H	H	H	H	H	H	H	H
L	L	L	H	H	L	H	H	H	H	H	H	L	H	H	H	H	H	H	H	H	H
L	L	L	H	H	H	H	H	H	H	H	H	H	L	H	H	H	H	H	H	H	H
L	L	H	L	L	L	H	H	H	H	H	H	H	H	L	H	H	H	H	H	H	H
L	L	H	L	L	H	H	H	H	H	H	H	H	H	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	H	H	H	H	H	H	H	H	L	H	H	H	H	H
L	L	H	L	H	H	H	H	H	H	H	H	H	H	H	H	H	L	H	H	H	H
L	L	H	H	L	L	H	H	H	H	H	H	H	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	H	H	H	H	H	H	H	H	L	H	H
L	L	H	H	H	L	H	H	H	H	H	H	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	L
L	H	X	X	X	X	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H
H	L	X	X	X	X	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H
H	H	X	X	X	X	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H	H

H = High Voltage Level, L = Low Voltage Level, X = Don't Care

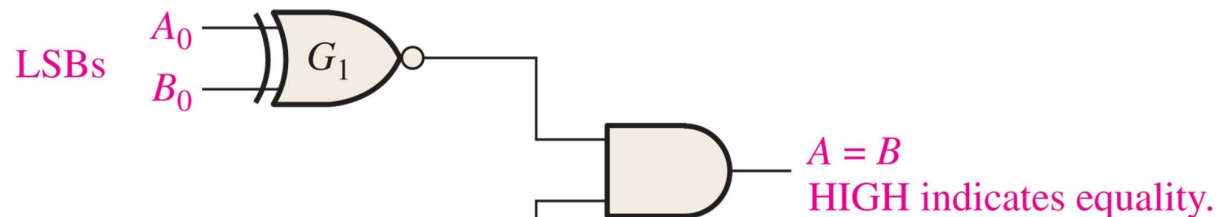


Comparators (Equality)

The function of a comparator is to compare the magnitudes of two binary numbers to determine the relationship between them. In the simplest form, a comparator can test for equality using XNOR gates.



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

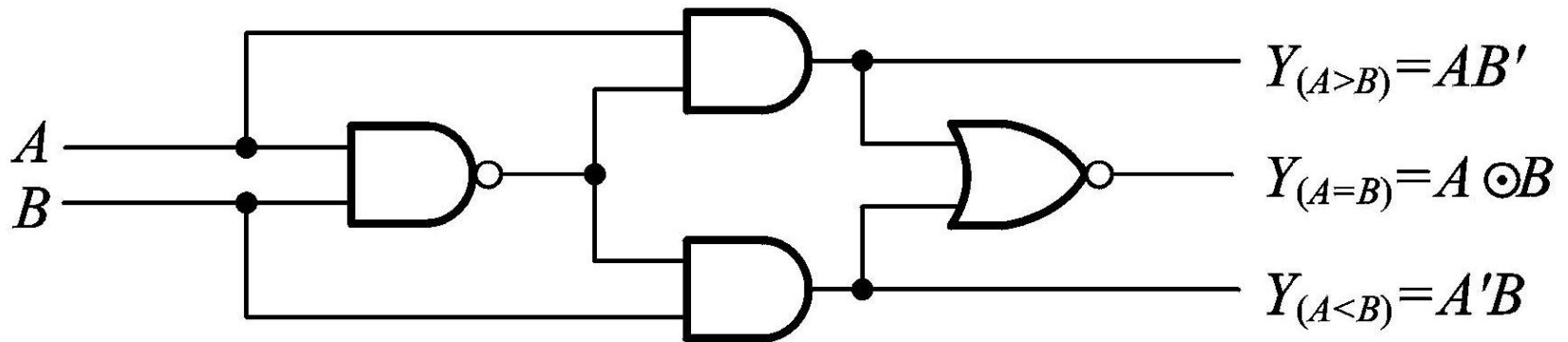


General format: Binary number $A \rightarrow A_1A_0$
Binary number $B \rightarrow B_1B_0$



Comparators (with Inequality)

Diagram of 1 bit comparator



Multiple bits: Compare from high bit to low bit

1. For $Y(C > D)$:

$$Y(C > D) = C_n \overline{D_n} + (C_n \odot D_n) \overline{C_{n-1} D_{n-1}} + \cdots + (C_n \odot D_n) (C_{n-1} \odot D_{n-1}) \cdots (C_1 \odot D_1) \overline{C_0 D_0}$$

2. For $Y(C < D)$:

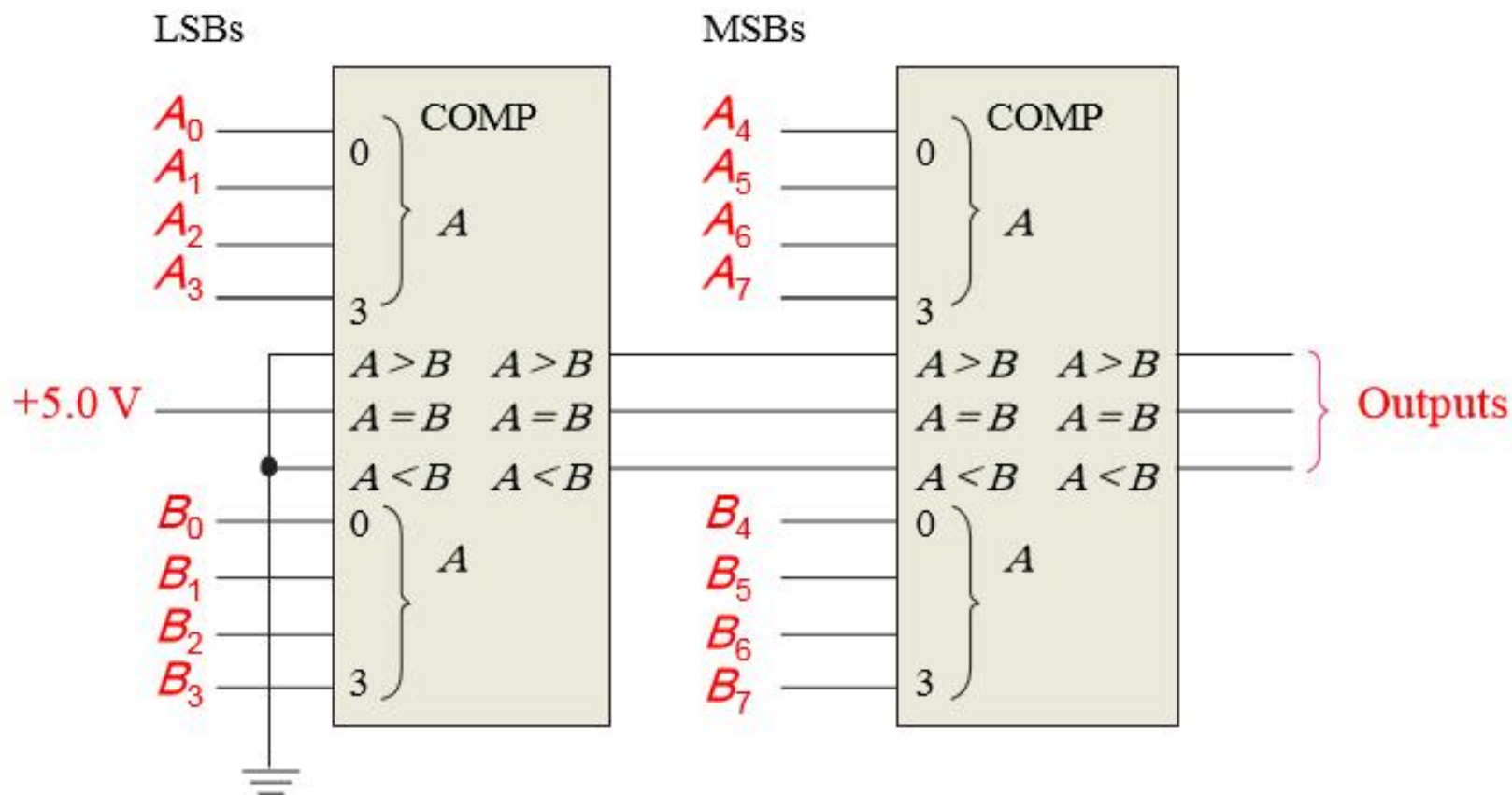
$$Y(C < D) = \overline{C_n} D_n + (C_n \odot D_n) \overline{C_{n-1} D_{n-1}} + \cdots + (C_n \odot D_n) (C_{n-1} \odot D_{n-1}) \cdots (C_1 \odot D_1) \overline{C_0 D_0}$$

3. For $Y(C = D)$:

$$Y(C = D) = (C_n \odot D_n) (C_{n-1} \odot D_{n-1}) \cdots (C_1 \odot D_1) (C_0 \odot D_0)$$

Numerical Comparator (Cascaded)

IC comparators can be expanded using the cascading inputs as shown. The lowest order comparator has a HIGH on the $A = B$ input.



Numerical Comparator (Cascaded)

1. **A and B: 4 highest-order bits of numbers with ≥ 4 bits**
2. **Results of lower-order bits (carry): $I_{(A>B)}$, $I_{(A<B)}$, and $I_{(A=B)}$**
3. **For numbers with ≤ 4 bits (first stage of cascaded structure):**
 $I_{(A>B)}=I_{(A<B)}=0$, $I_{(A=B)}=1$

1. **For $Y(A > B)$:**

$$\begin{aligned} Y(A > B) = & A_3 B'_3 + (A_3 \odot B_3) A_2 B'_2 + (A_3 \odot B_3) (A_2 \odot B_2) A_1 B'_1 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) A_0 B'_0 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) (A_0 \odot B_0) I_{(A>B)} \end{aligned}$$

2. **For $Y(A < B)$:**

$$\begin{aligned} Y(A < B) = & A'_3 B_3 + (A_3 \odot B_3) A'_2 B_2 + (A_3 \odot B_3) (A_2 \odot B_2) A'_1 B_1 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) A'_0 B_0 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) (A_0 \odot B_0) I_{(A<B)} \end{aligned}$$



Multiplexers (Data Selectors)

A multiplexer (MUX) selects one data line from two or more input lines and routes data from the selected line to the output. The particular data line that is selected is determined by the select inputs.

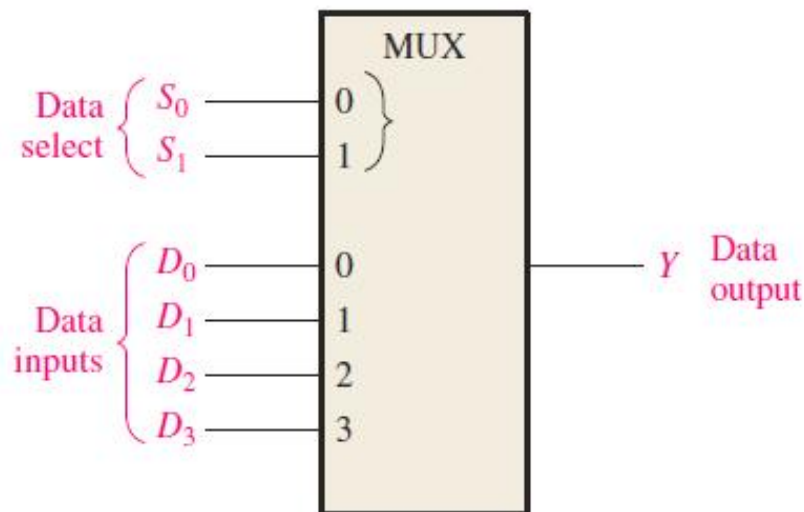


TABLE 6-8

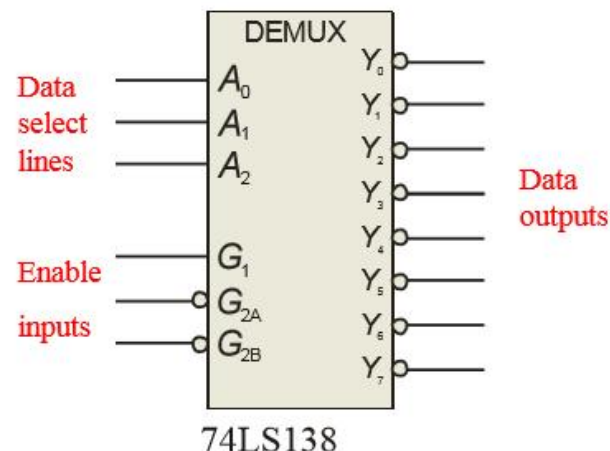
Data selection for a 1-of-4-multiplexer.

Data-Select Inputs		Input Selected
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$Y = D_0\bar{S}_1\bar{S}_0 + D_1\bar{S}_1S_0 + D_2S_1\bar{S}_0 + D_3S_1S_0$$

Demultiplexers (Data Distributors)

The 74LS138 was introduced previously as a decoder but can also serve as a DEMUX. When connected as a DEMUX, data is applied to one of the enable inputs, and routed to the selected output line depending on the select variables. Note that the outputs are active-LOW as illustrated in the following example...



INPUTS						OUTPUTS							
STROBE			ADDRESS										
G2	G1	G0	A ₂	A ₁	A ₀	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
X	X	H	X	X	X	H	H	H	H	H	H	H	H
L	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
H	L	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	L	H	H	H	H	H	L	H	H	H	H
H	L	L	H	L	L	H	H	H	H	L	H	H	H
H	L	L	H	L	H	H	H	H	H	H	L	H	H
H	L	L	H	H	L	H	H	H	H	H	H	L	H
H	L	L	H	H	H	H	H	H	H	H	H	H	L

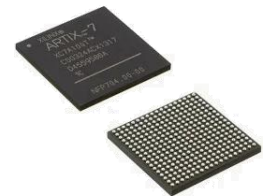


Combinational Logic

VHDL

FPGA

- Field Programmable Gate Array
- A type of integrated circuit (IC) that can be reconfigured after manufacturing
- Flexible for reconfiguration
- High performance for solving parallelable tasks.



CPLD: Complex programmable logic device

Compared to FPGA:

- Non-volatile configuration memory
- Less resources than FPGA (logic gates)
- Cost effective



VHDL vs Verilog

VHDL and Verilog are both Hardware Description Languages (HDLs) used for designing and simulating digital circuits

But they differ in syntax, verbosity, and typing. Verilog is more compact and C-like, while VHDL is more verbose and has a stricter, Ada-like syntax.

We will talk about VHDL first, and then Verilog



Basic constructs (building blocks)

- Entity

 - Analogy: circuit symbol

 - Define circuit interface

- Architecture

 - Analogy: circuit diagram

 - Define circuit functionality



VHDL code: overview

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity circuit1 is
    Port ( a : in  STD_LOGIC;
           b : in  STD_LOGIC;
           c : in  STD_LOGIC;
           f : out STD_LOGIC);
end circuit1;
```

→ Use a library

→ Use packages to provide various data types

→ Define an entity: “circuit1”

→ Define the entity ports: name, mode, and data type



VHDL code: overview

```
architecture behv1 of circuit1 is  
  -- this is a comment line  
  
  signal f1 : std_logic;  
  signal f2 : std_logic;  
  
begin  
  f1<=a and b;  
  f2<=c and NOT b;  
  f<=f1 or f2;  
end behv1;
```

Define an architecture
“behv1” for entity “circuit1”

Comment starts with “--”

Declare internal signals: f1 and f2

Define circuit operation



VHDL is concurrent

```
architecture behv1 of circuit1 is  
  
    -- this is a comment line  
  
    signal f1 : std_logic;  
    signal f2 : std_logic;  
  
begin  
    f1<=a and b;  
    f2<=c and NOT b;  
    f<=f1 or f2;  
end behv1;
```

These statements are **concurrent**.
They can be written in any order and
the circuit operation is the same.

VHDL is NOT case sensitive



VHDL code: overview

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity example1 is
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          c : out STD_LOGIC;
          s : out STD_LOGIC);
end example1;

architecture rtl of example1 is
begin
    c<=a and b;
    s<=a xor b
end rtl;
```



- Syntax

```
entity <entity_name> is
  port (
    <port_names> : <mode> <type>;
    <port_names> : <mode> <type>;
    ...
    -- last port has no semicolon
    <port_names> : <mode> <type>
  );
end <entity_name>;
```

- Example

```
ENTITY model1 IS
  --VHDL is case insensitive
  PORT( a :IN std_logic;
        b :IN std_logic;
        c,d :IN std_logic;
        e :OUT std_logic);
END model1 ;
```



Entity: port mode

<mode>: indicate the port direction

- **IN**: port value may only be read
- **OUT**: port value may be updated
- **INOUT**: data can flow both in and out of the entity, and the signal can be driven from outside the entity.
 - Used when there are multiple drives, such as tri state buffers.
- **BUFFER**: an output, but can be read inside the entity. (can be on both left and right hand side of $\leq$)

Entity: port type

<type>: indicate the signal type of the port

- **bit**: 0 or 1
- **bit_vector**: vector of **bit** values
- **Boolean**: TRUE or FALSE
- **integer**: integers

→ Part of VHDL standard

- **std_logic**: values for signal value and strength
- **std_logic_vector**: vector of **std_logic** values

→ **Require
IEEE
package**

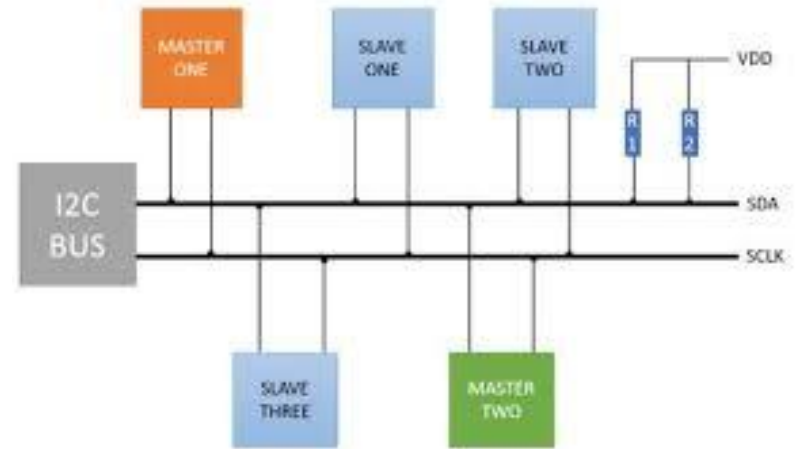
* integer supports calculation. bit_vector does not.



Entity: port type (std_logic)

IEEE 'std_logic' (package: 'std_logic_1164')

- '0': binary zero
- '1': binary one
- 'Z': high impedance
- 'U': uninitialized
- 'X': unknown
- ...



```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
ENTITY modell IS  
    PORT( a :IN std_logic;  
          b :IN std_logic;  
          c :IN std_logic;  
          d :IN std_logic;  
          e:OUT std_logic_vector(7 downto 0));  
END modell ;
```



Entity: port type (bit_vector)

Example

```
ENTITY model1 IS
    PORT( a :IN bit;
          b :IN bit;
          c :IN bit;
          d :IN bit;
          e :OUT bit_vector(7 downto 0) );
END model1 ;
```



Entity: port type (std_logic_vector)

bit_vector: array of bit objects

std_logic_vector: array of std_logic objects

Example

```
ENTITY model1 IS
    PORT( a :IN std_logic_vector(1 to 4);
          b :OUT std_logic_vector(7 downto 0);
          c :OUT std_logic_vector(1 to 4));
END model1 ;
```

Results

- a <= "0011": a(1)=0, a(2)=0, a(3)=1, a(4)=1
- b <= "10011000": b(7)=1, b(6)=0, ..., b(0)=0
- c <= "1011": c(1)=1, c(2)=0, c(3)=1, c(4)=1



Architecture: overview

An **architecture** represents one possible implementation of its associated entity

- **architecture declarations**: define internal signals, components, etc. to be used in architecture body
- **architecture body**: define implementation details of input/output relationship

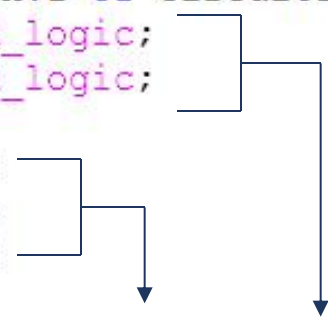


Architecture: overview

- Syntax

```
architecture <arch_name> of <entity_name> is  
    -- architecture declarations  
begin  
    -- architecture body  
end <arch_name>;
```

```
architecture behv1 of circuit1 is  
    signal f1 : std_logic;  
    signal f2 : std_logic;  
begin  
    f1<=a and b;  
    f2<=c and d;  
    f<=f1 or f2;  
end behv1;
```



CONCURRENT
ASSIGNMENT
statements

SIGNAL
declarations



Architecture:overview

Multiple architectures can exist for an entity.

(But only one can be active)

- Behavioral Level: what the system does, not how it is implemented.
- RTL (Register Transfer Level): Describes how the data moves and what logic processes it
(**synthesizable**: can be converted to gate-level netlists)
- Gate Level: Describes the circuit in terms of logic gates (AND, OR, NOT, flip-flops, etc.).



Architecture:overview

Entity can have multiple architectures

```
architecture RTL of MyEntity is
begin
    y <= a and b;
end RTL;

architecture Behavioral of MyEntity is
begin
    process(a, b)
    begin
        if a = '1' and b = '1' then
            y <= '1';
        else
            y <= '0';
        end if;
    end process;
end Behavioral;
```

(Define RTL and Behavioral arch)

```
architecture Testbench of MyTest is
    component MyEntity
        Port (
            a : in std_logic;
            b : in std_logic;
            y : out std_logic
        );
    end component;

    signal a, b, y : std_logic;

begin
    UUT: MyEntity
        port map (
            a => a,
            b => b,
            y => y
        );

    -- Configuration
    for UUT: MyEntity use entity work.MyEntity(RTL); -- specify architecture
end Testbench;
```

(Active RTL architecture)



Architecture declarations: **signal**

Used in **architecture declarations**.

a signal is a fundamental object used to represent connections (wires) between hardware elements.

Syntax: **signal** <signal_name> : <type>

```
ARCHITECTURE expr OF example1 IS
  SIGNAL u, w, x, y, z :std_logic;
  SIGNAL a, b, c :integer;
BEGIN
  x <= y AND z; -- logical expression
  w <= NOT x;
  u <= w;        -- direct assignment
  c <= a + b;    -- arithmetic expression
END expr;
```

```
signal x1,x2,Cin,Cout,Sel      : std_logic;
signal C                      : std_logic_vector (1 to 4);
signal X,Y,S                  : std_logic_vector (3 downto 0);
```

General Form

```
architecture <arch_name> of <entity_name> is
  -- architecture declarations
  [SIGNAL declarations]
  [COMPONENT declarations]
  [CONSTANT declarations]
  [TYPE declarations]
  [ATTRIBUTE specifications]
begin
  -- architecture body
  [CONCURRENT ASSIGNMENT
   statements;]
  [COMPONENT instantiation statements;]
  [PROCESS statements;]
  [GENERATE statements;]
end <arch_name>;
```



Simple Assignment: direct signal assignment

Used in **architecture body**.

Syntax: <target> <= <expression>

- <target> can be an internal signal or an output port
- <expression> operates on internal signals and/or input ports

```
ARCHITECTURE expr OF example1 IS
SIGNAL u, w, x, y, z :std_logic;
SIGNAL a, b, c :integer;
BEGIN
    x <= y AND z; -- logical expression
    w <= NOT x;
    u <= w;        -- direct assignment
    c <= a + b;    -- arithmetic expression
END expr;
```

The assignment is **concurrent**

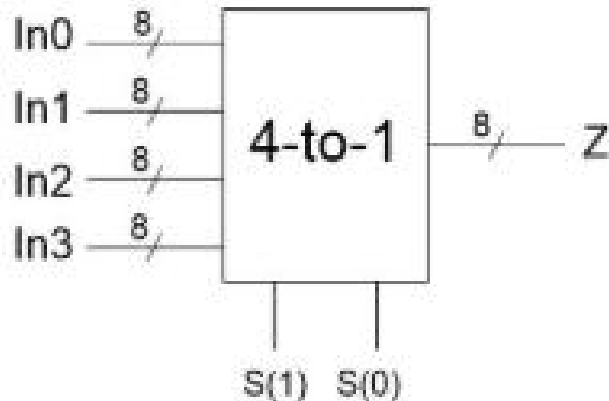
General Form

```
architecture <arch_name> of <entity_name> is
    -- architecture declarations
    [SIGNAL declarations]
    [COMPONENT declarations]
    [CONSTANT declarations]
    [TYPE declarations]
    [ATTRIBUTE specifications]
begin
    -- architecture body
    [CONCURRENT ASSIGNMENT
     statements;]
    [COMPONENT instantiation statements;]
    [PROCESS statements;]
    [GENERATE statements;]
end <arch_name>;
```



Conditional Assignment: When else

A signal can be assigned one of several values based on a **conditional criterion**.



```
architecture con_arch1 of mux4 is
begin
    Z <= In0 when S = "00" else
        In1 when S = "01" else
        In2 when S = "10" else
        In3;
end architecture con_arch1;
```

Conditional Assignment: With Select

A signal can be assigned one of several values based on a **selection criterion**.

Syntax

```
with <select_signal> select
<target> <= <expression> when <value>,
        <expression> when <value>,
        ....
        < expression> when others;
```

Example

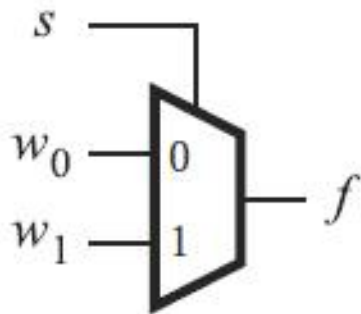
```
architecture behv of mux
begin
  with s select
    f <= a when "00",
        b when "01",
        c when "10",
        d when others;
end behv;
```



Example: 2-to-1 mux (ch. 6)

s	f
0	w_0
1	w_1

(b) Truth table



(a) Graphical symbol

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY mux2to1 IS
    PORT ( w0, w1, s : IN  STD_LOGIC ;
          f          : OUT STD_LOGIC ) ;
END mux2to1 ;

ARCHITECTURE Behavior OF mux2to1 IS
BEGIN
    WITH s SELECT
        f <=  w0 WHEN '0',
              w1 WHEN OTHERS ;
END Behavior ;
```

Figure 6.27 VHDL code for a 2-to-1 multiplexer.



Operators

Order of precedence

	Class	Operator
Highest	Miscellaneous	NOT, abs, ** (exponentiation)
	Multiplying	*, /, mod (modulus), rem (remainder)
	Sign	+, -
	Adding	+, -, & (concatenation)
	Shift	sll, srl, sla, sra, rol, ror
	Relational	=, /=, <, <=, >, >=
Lowest	Logical	AND, OR, NAND, NOR, XOR, XNOR



Component

An **entity** defined in one source code file can be used as a **component** in another source code file.

Requires component declaration and component instantiation statements.

```
architecture <arch_name> of <entity_name> is
    -- architecture declarations
    [SIGNAL declarations]
    [COMPONENT declarations]
    [CONSTANT declarations]
    [TYPE declarations]
    [ATTRIBUTE specifications]
begin
    -- architecture body
    [CONCURRENT ASSIGNMENT statements;]
    [COMPONENT instantiation statements;]
    [PROCESS statements;]
    [GENERATE statements;]
end <arch_name>;
```



Component

Three steps:

- (1) Design full adder.
- (2) Declare full adder in architecture.
- (3) Instantiate full adder in architecture's body.

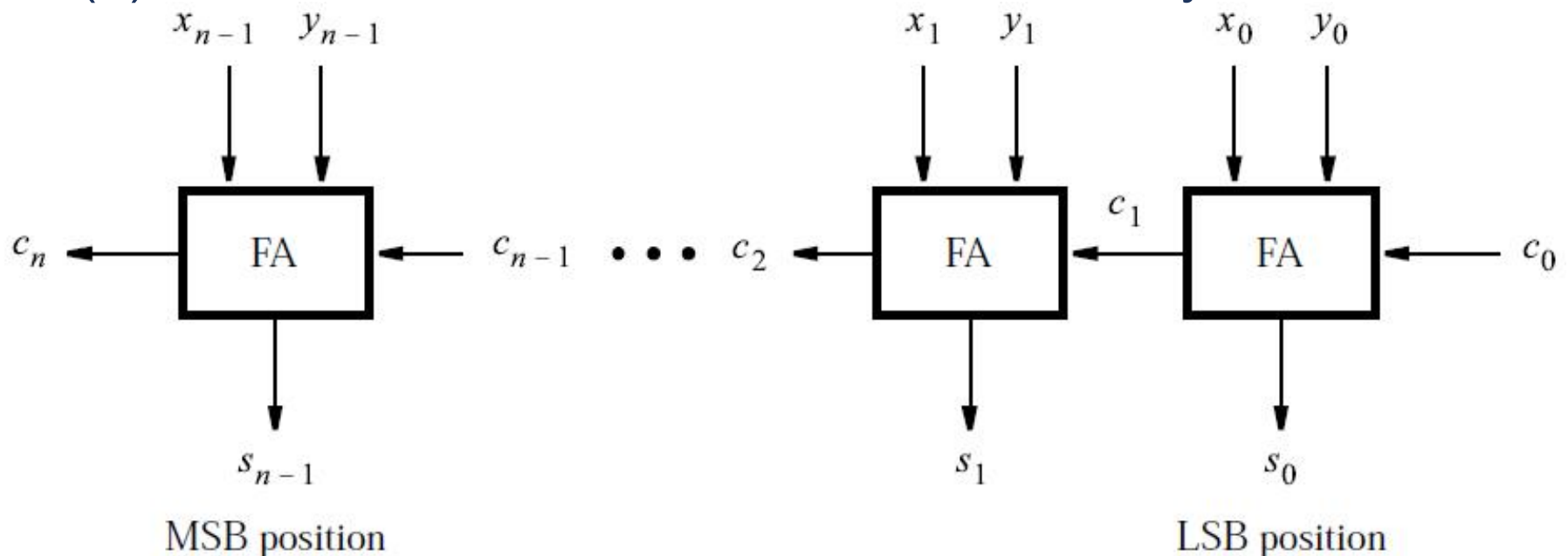


Figure 5.6 An n -bit ripple-carry adder.

Example: implement a multi-bit adder.

First, use VHDL code for 1-bit full adder

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY fulladd IS
    PORT ( Cin, x, y : IN    STD_LOGIC ;
          s, Cout    : OUT  STD_LOGIC ) ;
END fulladd ;

ARCHITECTURE LogicFunc OF fulladd IS
BEGIN
    s <= x XOR y XOR Cin ;
    Cout <= (x AND y) OR (Cin AND x) OR (Cin AND y) ;
END LogicFunc ;
```



Component

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY adder4 IS
    PORT ( Cin          : IN    STD_LOGIC ;
          x3, x2, x1, x0 : IN    STD_LOGIC ;
          y3, y2, y1, y0 : IN    STD_LOGIC ;
          s3, s2, s1, s0 : OUT   STD_LOGIC ;
          Cout           : OUT   STD_LOGIC ) ;
```

```
END adder4 ;
```

```
ARCHITECTURE Structure OF adder4 IS
```

```
    SIGNAL c1, c2, c3 : STD_LOGIC ;
```

```
    COMPONENT fulladd
```

```
        PORT ( Cin, x, y : IN    STD_LOGIC ;
              s, Cout    : OUT   STD_LOGIC ) ;
```

```
    END COMPONENT ;
```

```
BEGIN
```

```
    stage0: fulladd PORT MAP ( Cin, x0, y0, s0, c1 ) ;
    stage1: fulladd PORT MAP ( c1, x1, y1, s1, c2 ) ;
    stage2: fulladd PORT MAP ( c2, x2, y2, s2, c3 ) ;
    stage3: fulladd PORT MAP (
        Cin => c3, Cout => Cout, x => x3, y => y3, s => s3 ) ;
```

```
END Structure ;
```

declaration

Positional association

instantiation

named association



PROCESS

- Previous VHDL code executes concurrently like hardware does.
- But sometimes, you need to describe sequential operations.
 - Things require step-by-step updates based on a clock (like registers, counters, state machines), which is intractable to be modeled as concurrent assignments.
 - Difficult to describe "only update on rising edge of clk" without process.
- Process: allows you to write sequential logic inside a concurrent framework



PROCESS

- Multiple PROCESS statements are concurrent
- Statements inside a PROCESS are sequential

```
[process_label:]  
PROCESS (<signal1>, <signal2> ...)  
    [VARIABLE declarations]  
BEGIN  
    [WAIT statement]  
    [IF statements]  
    [CASE statements]  
    [FOR-LOOP statements]  
    [WHILE-LOOP statements]  
END PROCESS;
```



When PROCESS executes?

- In simulation, processes execute once at time 0.
- In synthesized hardware, there is no such initial execution unless you define it explicitly (e.g., via a reset signal).

```
process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            q <= '0'; -- Reset value
        else
            q <= d;    -- Normal operation
        end if;
    end if;
end process;
```



When PROCESS executes?

Process executes when signal values in sensitivity list got changed.

Optional: process_label

Sensitivity list: (<signal 1>, <signal 2> ...)

- Specify signals that cause the PROCESS to be evaluated
- When the value of any signal in the sensitivity list changes, the PROCESS is **active** and the statements inside the PROCESS are executed in sequential order
- Statements inside PROCESS: IF, CASE, WAIT, LOOP

```
[process_label:]  
PROCESS (<signal1>, <signal2> ...)   
    [VARIABLE declarations]  
BEGIN  
    [WAIT statement]  
    [IF statements]  
    [CASE statements]  
    [FOR-LOOP statements]  
    [WHILE-LOOP statements]  
END PROCESS;
```



When PROCESS executes?

The sensitivity list is a list of signal to which the process responds.

```
signal a, b, c, y: std_logic; -- in architecture declaration
...
process (a, b, c) -- in architecture body
begin
    y <= a and b and c; -- infer a combinational circuit
end process;
```

```
signal a, b, c, y: std_logic;
...
process (a) -- a process with incomplete sensitivity list
    -- not infer a combinational circuit
begin
    y <= a and b and c;
end process;
```



When PROCESS executes?

Wait statement: pause or suspend the execution of a process until a certain condition is met.

```
process
begin
    y <= a and b and c;
    wait on a, b, c;
end process;
```

Statement

Synthesizable?

wait until rising_edge(clk);

✓ Yes

wait for 10 ns;

✗ No

wait on a, b, c;

⚠ Tool-dependent



When PROCESS executes?

Compare wait with sensitivity list?



```
process (a, b) is  
begin  
  c <= a and b;  
end process;
```

sensitivity list. The process will be activated if either a or b (or both) changes states

```
Process is  
begin  
  c <= a and b;  
  wait on a, b;  
end process;
```

A process can have wait statement or sensitivity list, but not both

What happens if a process has neither a wait statement nor a sensitivity list?

PROCESS: Variables

- Variables can be declared and used inside a process statement.
- Variable assignment statement: `variable-object := expression;`
- Variable assignment is immediate.

```
Signal a, b, y : std_logic;  
*****  
process (a, b) is  
    variable temp : std_logic;  
    begin  
        temp := '0';  
        temp := temp or a;  
        temp := temp or b;  
        y <= temp;  
    end process;
```

Although the behavior of a variable is easy to understand, mapping it to hardware is difficult. **Not recommended.** We should try to use signal in code in general and resort to variable only for the characteristic that cannot be described by signals.



PROCESS: IF statement

IF statement can only appear in a PROCESS Syntax

```
if <condition> then
    statements;
[elsif <condition> then
    statement; ]
[else
    statement; ]
end if;
```

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

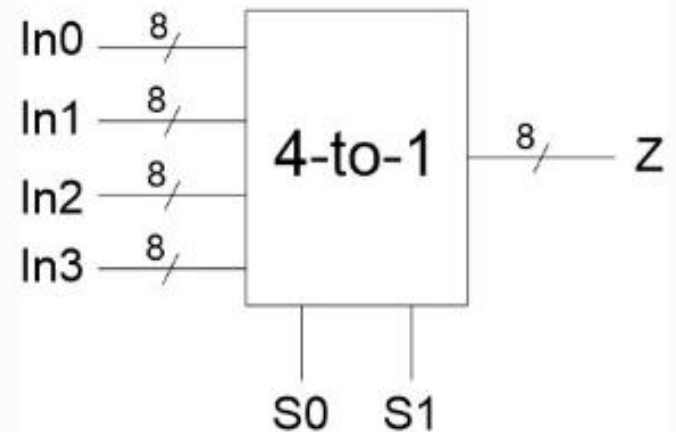
ENTITY mux2to1 IS
    PORT ( w0, w1, s : IN    STD_LOGIC ;
          f          : OUT  STD_LOGIC ) ;
END mux2to1 ;

ARCHITECTURE Behavior OF mux2to1 IS
BEGIN
    PROCESS ( w0, w1, s )
    BEGIN
        f <= w0 ;
        IF s = '1' THEN
            f <= w1 ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

PROCESS: IF statement

4-to-1 multiplexer

```
architecture if_arch of mux4 is
begin
  process(In0, In1, In2, In3, S)
  begin
    if (S = "00") then
      Z <= In0;
    elsif (S = "01") then
      Z <= In1;
    elsif (S = "10") then
      Z <= In2;
    else Z <= In3;
    end if;
  end process;
end if_arch;
```



PROCESS: IF statement

Clock behaviours: rising and falling.
(Can only be used within process.)

```
architecture Behavioral of MyEntity is
    signal clk : std_logic;
    signal d   : std_logic;
    signal q   : std_logic;
begin

    process(clk)
    begin
        if rising_edge(clk) then
            q <= d;
        end if;
    end process;

end Behavioral;
```

```
architecture Behavioral of MyEntity is
    signal clk : std_logic;
    signal d   : std_logic;
    signal q   : std_logic;
begin

    process(clk)
    begin
        if falling_edge(clk) then
            q <= d;
        end if;
    end process;

end Behavioral;
```



PROCESS: CASE statement

CASE statement can only appear in a PROCESS

Syntax

```
case <expression> is
    when <value> =>
        statements;
    when <value> =>
        statements;
    [ when others =>
        statements; ]
end case;
```

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

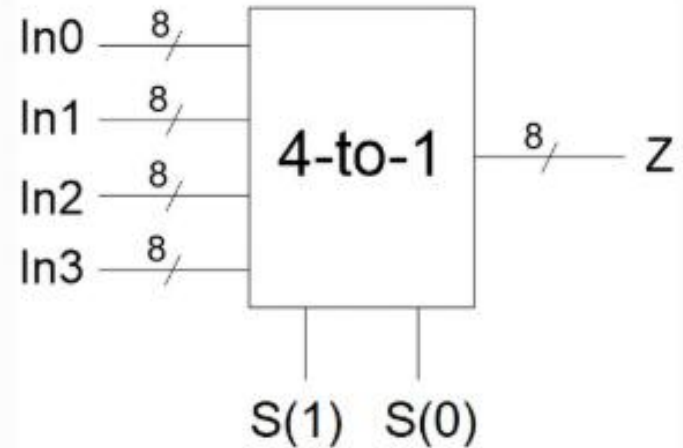
ENTITY mux2to1 IS
    PORT ( w0, w1, s : IN  STD_LOGIC ;
          f          : OUT STD_LOGIC ) ;
END mux2to1 ;

ARCHITECTURE Behavior OF mux2to1 IS
BEGIN
    PROCESS ( w0, w1, s )
    BEGIN
        CASE s IS
            WHEN '0' =>
                f <= w0 ;
            WHEN OTHERS =>
                f <= w1 ;
        END CASE ;
    END PROCESS ;
END mux2to1 ;
```



PROCESS: CASE statement

```
architecture case_arch of mux4 is
begin
  process(In0, In1, In2, In3, S)
  begin
    case S is
      when "00" =>
        Z <= In0;
      when "01" =>
        Z <= In1;
      when "10" =>
        Z <= In2;
      when others
        Z <= In3;
    end case;
  end process;
end case_arch;
```



PROCESS: CASE statement

When's value can indicate multiple entries.

```
architecture case_arch of pr_encoder is
begin
  process(S)
  begin
    case S is
      when "1000" | "1001" | "1010" | "1011"
         "1100" | "1101" | "1110" | "1111"
        Z <= "11";
      when "0100" | "0101" | "0110" | "0111"
        Z <= "10";
      when "0010" | "0011" =>
        Z <= "01";
      when others
        Z <= "00";
    end case;
  end process;
end case_arch;
```



FOR LOOP

In PROCESS, to repeat a set of statements a specific number of times

```
for i in 0 to N loop
    -- statements
end loop;
```

```
for i in N downto 0 loop
    -- statements
end loop;
```

```
process(clk)
begin
    if rising_edge(clk) then
        for i in 0 to 7 loop
            data_out(i) <= not data_in(i);
        end loop;
    end if;
end process;
```



FOR LOOP

Synthesizable if loop range is determined at compile time, and the content of loop is synthesizable.

Example: calculate parity using a loop

```
architecture rtl of parity_check is

    signal r_data          : std_logic_vector(7 downto 0);

begin

    p_parity_check : process (i_clk)
        variable vparity    : std_logic;
    begin
        if rising_edge(i_clk) then
            r_data <= i_data;

            vparity := '0';
            l_parity : for k in 0 to r_data'length-1 loop
                vparity := vparity xor r_data(k);
            end loop l_parity;

            o_parity <= vparity;
        end if;
    end process p_parity_check;

end rtl;
```



While Loop

instead of looping over a fixed range, this loops as long as a condition is true.

```
while condition loop
    -- sequential statements
end loop;
```

Often not synthesizable if
the number of iterations
isn't determinable at
compile time

```
process(clk)
    variable i : integer;
begin
    if rising_edge(clk) then
        i := 0;
        while i < 4 loop
            data_out(i) <= not data_in(i);
            i := i + 1;
        end loop;
    end if;
end process;
```



Testbenches consist of non-synthesizable VHDL code which generate inputs to the design and checks that the outputs are correct.

reset <= '1', '0' after 1 us;

clock <= not clock after 10 ns;

wait for 10 ns;

wait until (reset = '0'); (wait for reset=0)

wait; (wait forever)

wait on sig_a, sig_b;

(waits for a change of state in either the sig_a or sig_b signals.)

assert(q_out='0') report "Fail 0/0" severity error;

(If the condition is **false**, it triggers a simulation warning or error. report when q_out is 1.)

assert false report "test done";

report <message_string> [severity <severity_level>];

```
report "this is a message"; -- severity note  
-- or:  
report "this is a serious message" severity warning;
```

```
assert i < 5 report "unexpected value. i = " & integer'image(i);
```

If $i=6$, then report: "unexpected value. $i = 6$ "

- Severity can be: note, warning, error, failure.
- image attribute: convert integer into a string representation.



Testbench example

```
1 entity example_tb is
2 end entity example_tb;
3
4 architecture test of example_tb is
5   signal clock : std_logic := '0';
6   signal reset : std_logic := '1';
7   signal and_in : std_logic_vector(1 downto 0) := (others => '0');
8   alias in_a is and_in(0);
9   alias in_b is and_in(1);
10  signal out_q : std_logic;
11 begin
12   -- Reset and clock
13   clock <= not clock after 1 ns;
14   reset <= '1', '0' after 5 ns;
15
16   -- Instantiate the design under test
17   dut: entity work.example_design(rtl)
18   port map (
19     clk => clock,
20     reset => reset,
21     a => in_a,
22     b => in_b,
23     q => out_q
24   );
25
```

instantiate: entity
“example_design” in
library “work”. Use rtl
architecture.

```
26 -- Generate the test stimulus
27 stimulus:
28 process begin
29   -- Wait for the Reset to be released before
30   wait until (reset = '0');
31   -- Generate each of in turn, waiting 2 clock periods between
32   -- each iteration to allow for propagation times
33   and_in <= "00";
34   wait for 2 ns;
35   and_in <= "01";
36   wait for 2 ns;
37   and_in <= "10";
38   wait for 2 ns;
39   and_in <= "11";
40   -- Testing complete
41   wait;
42 end process stimulus;
43
44 end architecture test;
```

This generate signals
driving the “example_design”

This is called “direct entity instantiation”.

Use “entity” for instantiation rather than “component”.
“component” is usable but outdated style for testbench.



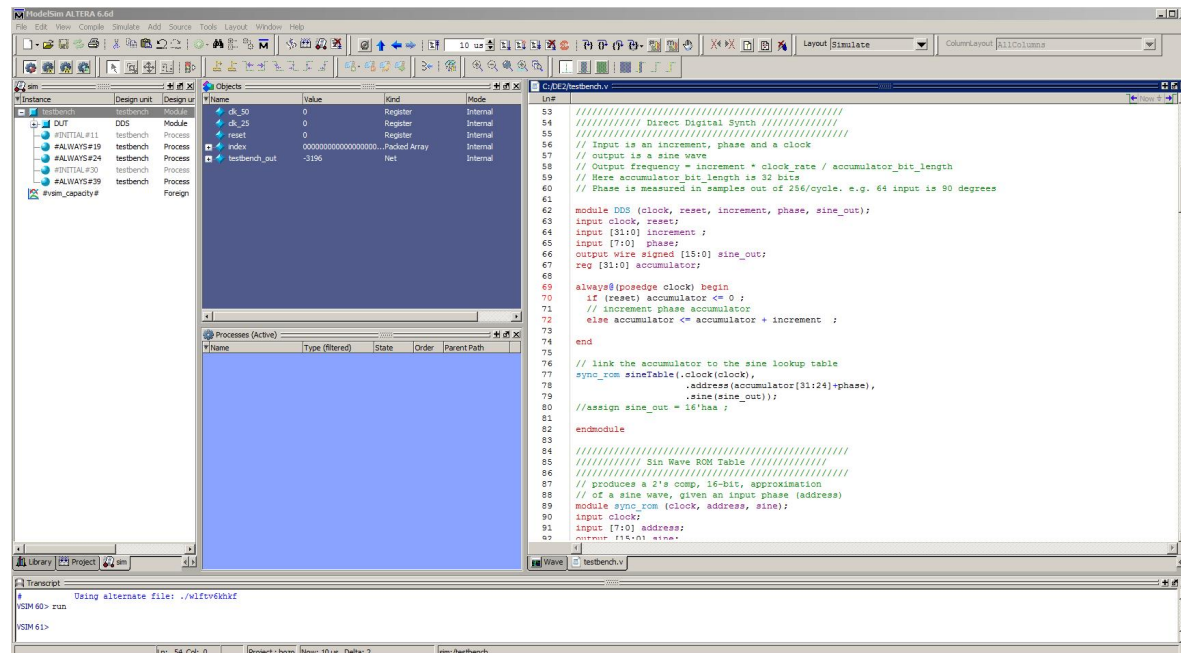
VHDL Resources

Online simulator: <https://edaplayground.com/>

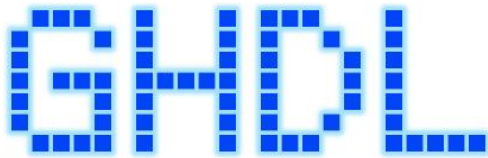
Tutorial: https://web.cs.ucla.edu/Logic_Design/vhdlintro.htm

<https://sustechvhdl.readthedocs.io/lecture/chapter1.html>

ModelSim/Quartus II (Altera)/ Vivado (Xilinx)



VHDL Resources

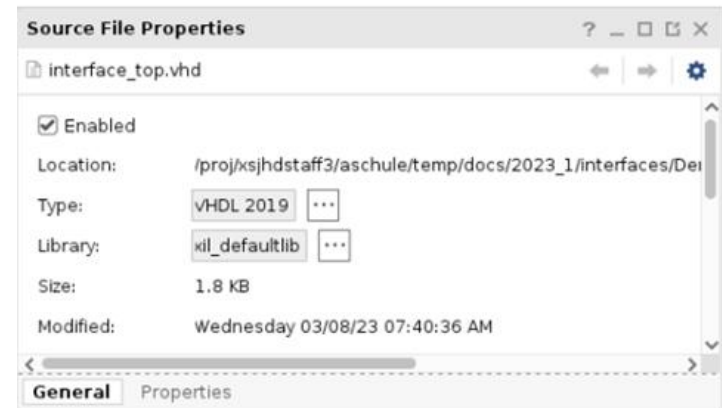
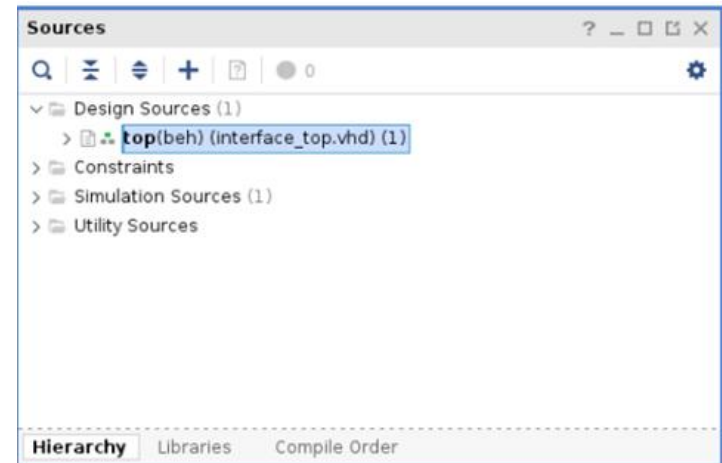
The logo for GHDL, consisting of the letters 'GHDL' in a blue, pixelated, blocky font.

ghdl.github.io/ghdl up | Chat on gitter Docker 75k commits since v5.0.1 55

Test failing Test failing CII 149%

This directory contains the sources of GHDL, the open-source analyzer, compiler, simulator and (experimental) synthesizer for [VHDL](#), a Hardware Description Language ([HDL](#)). GHDL is not an interpreter: it allows you to analyse and elaborate sources for generating machine code from your design. Native program execution is the only way for high-speed simulation.

<https://github.com/ghdl/ghdl>



AMD Vivado

<https://docs.amd.com/r/en-US/ug901-vivado-synthesis/Setting-up-Vivado-to-use-VHDL-2019>



上海科技大学
ShanghaiTech University

Online VHDL simulator

EDA playground

New Run Save Copy

Brought to you by DOULOS

Doulos does not endorse training material from other suppliers on EDA Playground.

Languages & Libraries

Testbench + Design

VHDL

Libraries

None

OVL

OSVVM

Top entity

testbench

Enable VUnit

Tools & Simulators

Aldec Riviera Pro 2023.04

Compile Options

-2008

Run Options

Run Options

Run Time: 10 ms

Use run.do Tcl file

Use run.bash shell script

Open EPWave after run

Show output file after run

Download files after run

Examples

using EDA Playground

VHDL

Verilog/SystemVerilog

UVM

EaslerUVM

SVAUnit

SVUnit

VUnit (Verilog/SV)

VUnit (VHDL)

TL-Verilog

e + Verilog

Python + Verilog

Python Only

C++/SystemC

testbench.vhd

design.vhd

```
3 use IEEE.std_logic_1164.all;
4
5 entity testbench is
6   -- empty
7 end testbench;
8
9 architecture tb of testbench is
10
11   -- DUT component
12   component or_gate is
13     port(
14       a: in std_logic;
15       b: in std_logic;
16       q: out std_logic);
17   end component;
18
19   signal a_in, b_in, q_out: std_logic;
20
21 begin
22
23   -- Connect DUT
24   DUT: or_gate port map(a_in, b_in, q_out);
25
26   process
27   begin
28     a_in <= '0';
29     b_in <= '0';
30     wait for 1 ns;
31     assert(q_out='0') report "Fail 0/0" severity
32     error;
33
34     a_in <= '0';
35     b_in <= '1';
36     wait for 1 ns;
37     assert(q_out='1') report "Fail 0/1" severity
38     error;
39
40     a_in <= '1';
41     b_in <= 'X';
42     wait for 1 ns;
43     assert(q_out='1') report "Fail 1/X" severity
44     error;
45
46     a_in <= '1';
47     b_in <= '1';
48     wait for 1 ns;
49     assert(q_out='1') report "Fail 1/1" severity
50     error;
51
52     -- Clear inputs
53     a_in <= '0';
54     b_in <= '0';
55
56     assert false report "Test done." severity note;
57     wait;
58   end process;
59 end tb;
```

```
1 -- Simple OR gate design
2 library IEEE;
3 use IEEE.std_logic_1164.all;
4
5 entity or_gate is
6   port(
7     a: in std_logic;
8     b: in std_logic;
9     q: out std_logic);
10 end or_gate;
11
12 architecture rtl of or_gate is
13 begin
14   process(a, b) is
15   begin
16     q <= a or b;
17   end process;
18 end rtl;
```

Log

Share

```
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 5400 kB (elbread=427 elab2=4829 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# EXECUTION:: NOTE : Test done.
# EXECUTION:: Time: 4 ns, Iteration: 0, Instance: /testbench, Process: line_26.
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
```

Done

